

附件 3

西安电子科技大学

智能系统专业实验 课程实验报告

实验名称 实验 2-数据分类

XXXXXXXX 学院 2320xxx 班

姓名 XXXXXX 学号 XXXXXXXXXXXX

同作者 XXXXXXXXXXX

实验日期 202x 年 x 月 x 日

成 绩

实验报告内容基本要求及参考格式

- 一、实验目的
- 二、实验环境
- 三、实验基本原理及步骤（模型描述、理论描述或方案描述）
- 四、实验自主创新（或改进）部分的描述
- 五、实验结果与分析（需给出结果图或表，并进行分析）
- 六、实验的收获及心得体会（或遇到的问题，如何解决）
- 七、实验代码

一、实验目的

1. 掌握利用神经网络解决二分类问题的完整流程。
2. 学会使用 PyTorch 生成二维正态分布样本集，并构建全连接分类网络。
3. 理解 Softmax 输出与交叉熵损失函数的原理，并能计算分类准确率。
4. 能够独立编写训练程序和测试程序，并对测试样本进行详细的结果分析。

二、实验环境

1. 硬件：Nvidia Jetson TX 嵌入式开发板
2. 操作系统：Linux
3. 软件及库：
 - 1) Python 3
 - 2) NumPy
 - 3) PyTorch
 - 4) Matplotlib

三、实验基本原理及步骤（模型描述、理论描述或方案描述）

1. 基本原理

本实验为二分类问题，数据为二维平面上的点。

- 1) 第一类（标签 0）：均值 (2, 2)，方差 1 的正态分布。
- 2) 第二类（标签 1）：均值 (-2, -2)，方差 1 的正态分布。

采用三层全连接网络：

- 1) 输入层：2 个节点 (x, y 坐标)
- 2) 隐藏层：15 个节点，ReLU 激活函数
- 3) 输出层：2 个节点，Softmax 激活函数，输出属于两类的概率
- 4) 损失函数：交叉熵损失 (CrossEntropyLoss)。
- 5) 优化算法：随机梯度下降 (SGD)，学习率 0.03。

训练时，网络通过前向传播得到预测概率，与真实标签计算损失，反向传播更新权重。训练完成后，使用独立的测试集评估模型泛化能力。

2. 实验步骤

1) 构造训练样本集

- a) 生成 100 个点作为第一类 x_0 ；生成 100 个点作为第二类 x_1 ；
- b) 合并得到 x ，形状 (200, 2) 生成标签 y ：前 100 个 0，后 100 个 1，

2) 构建神经网络

- a) 输入维度：2
- b) 隐藏层：15 个神经元，ReLU 激活
- c) 输出维度：2

3) 训练网络

- a) 优化器：SGD，学习率 0.1
- b) 损失函数：交叉熵
- c) 迭代次数：100 次

4) 测试模型

- 生成 30 个新样本（15 个类别 0，15 个类别 1）作为测试集
- 使用训练好的模型进行预测，计算准确率
- 打印每个测试样本的真实标签、预测标签、类别概率及判断结果

5) 训练可视化与结果输出

- 动态散点图展示训练过程中分类边界的变化
- 控制台输出测试集详细结果

四、实验自主创新（或改进）部分的描述

1. 增加独立的测试程序

原始实验指导书仅要求训练并显示结果，本实验额外构建了 30 个新样本的测试集，对训练好的模型进行泛化能力评估。

2. 详细输出每个测试样本的预测概率

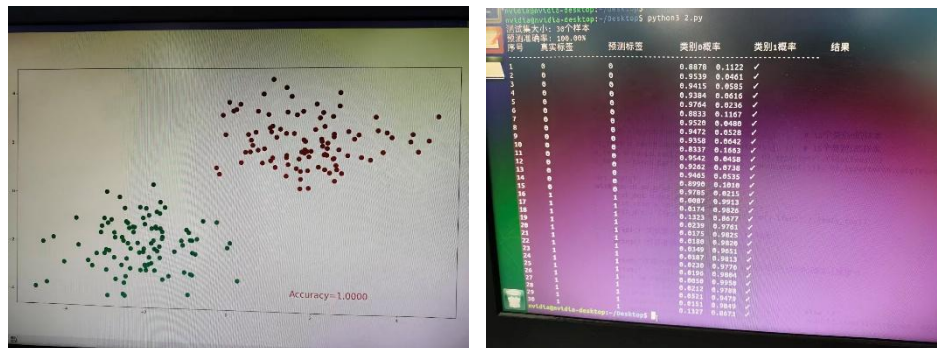
打印每个样本的类别 0 和类别 1 的概率，便于分析网络输出的置信度，以及判断哪些样本容易被误分类。

3. 不使用早停，完整训练 100 个 epoch

观察完整训练过程中准确率的变化，确保网络充分收敛，并验证最终准确率是否稳定在 100%。

五、实验结果与分析（需给出结果图或表，并进行分析）

1. 最终分类效果图



- 散点：红色点（预测类别 0）集中在右上角 (2,2) 附近，绿色点（预测类别 1）集中在左下角 (-2,-2) 附近。
- 分类曲线：分类边界为一条斜线，两类完全分开，无重叠。
- 图像上方显示 **Accuracy=1.0000**。

六、实验的收获及心得体会（或遇到的问题，如何解决）

- 掌握了分类任务的标准流程：**从数据生成、标签制作、网络构建、训练可视化到测试评估，形成了一套完整的代码框架，可以迁移到其他分类问题。
- 深入理解了 Softmax 与交叉熵：**Softmax 将网络输出转换为概率分布，便于解释和计算准确率，而交叉熵损失函数适合衡量两个概率分布的差异，是分类问题的首选损失函数。
- 遇到的小问题及解决：**
 - 初始代码中 `y1=torch.zeros(100)` 导致标签全为 0，训练无法收敛。通过检

查数据发现错误并修正为 `torch.ones(100)`。

- 2) 测试集生成时需保证前 15 个为类别 0、后 15 个为类别 1，使用切片 `test_data[:15]` 和 `test_data[15:]` 分别生成，避免混叠

七、实验代码

代码

```
import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt

# ===== 1. 构造训练样本集 =====
data = torch.ones(100, 2)
x0 = torch.normal(2 * data, 1)  # 类别 0, 均值 2
x1 = torch.normal(-2 * data, 1)  # 类别 1, 均值-2
x = torch.cat((x0, x1), 0).type(torch.FloatTensor)

y0 = torch.zeros(100)
y1 = torch.ones(100)
y = torch.cat((y0, y1), 0).type(torch.LongTensor)

# ===== 2. 定义网络 =====
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.classify = nn.Sequential(
            nn.Linear(2, 15),
            nn.ReLU(),
            nn.Linear(15, 2),
            nn.Softmax(dim=1)
        )
    def forward(self, x):
        return self.classify(x)

net = Net()
optimizer = torch.optim.SGD(net.parameters(), lr=0.03)
loss_func = nn.CrossEntropyLoss()

# ===== 3. 训练与动态可视化 =====
```

```

plt.ion()
for epoch in range(100):
    out = net(x)
    loss = loss_func(out, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if epoch % 5 == 0:
        plt.cla()
        classification = torch.max(out, 1)[1]
        class_y = classification.data.numpy()
        target_y = y.data.numpy()

        plt.scatter(x.data.numpy()[:, 0], x.data.numpy()[:, 1],
                    c=class_y, s=100, cmap="RdYlGn")
        accuracy = sum(class_y == target_y) / 200
        plt.text(1.5, -4, "Accuracy={:.4f}".format(accuracy),
                fontdict={"size": 20, "color": "red"})
        plt.pause(0.5)

plt.ioff()
plt.show()

# ===== 4. 测试集评估 =====
test_data = torch.ones(30, 2)
x0_test = torch.normal(2 * test_data[:15], 1)  # 15 个类别 0
x1_test = torch.normal(-2 * test_data[15:], 1)  # 15 个类别 1
x_test = torch.cat((x0_test, x1_test), 0).type(torch.FloatTensor)
y_test = torch.cat((torch.zeros(15), torch.ones(15)), 0).type(torch.LongTensor)

net.eval()
with torch.no_grad():
    test_out = net(x_test)
    test_pred = torch.max(test_out, 1)[1]
    test_acc = (test_pred == y_test).sum().item() / len(y_test)

    print(f"测试集大小: {len(y_test)} 个样本")

```

```
print(f'预测准确率: {test_acc:.2%}')
print("序号\t 真实标签\t 预测标签\t 类别 0 概率\t 类别 1 概率\t 结果")
print("-" * 70)
for i in range(len(y_test)):
    prob0 = test_out[i][0].item()
    prob1 = test_out[i][1].item()
    result = "✓" if test_pred[i] == y_test[i] else "X"

print(f'{i+1}\t{y_test[i].item()}\t\t{test_pred[i].item()}\t\t{prob0:.4f}\t{prob1:.4f}\t{
result}')
```